E01 Particle Effects

CRE132 worksheet

What you'll make

Little puffs of dust at the player's feet, a spray of yellow sparks when you hit a slime, an explosion of orange squares that fade to grey smoke when a barrel breaks, and a sparkle where a coin or gem was picked up.

By the end you'll know what a **Particle System** is and which of its settings matter most, how to make a one-off effect that cleans itself up, and how to trigger effects from Health's events without writing new code in Health.

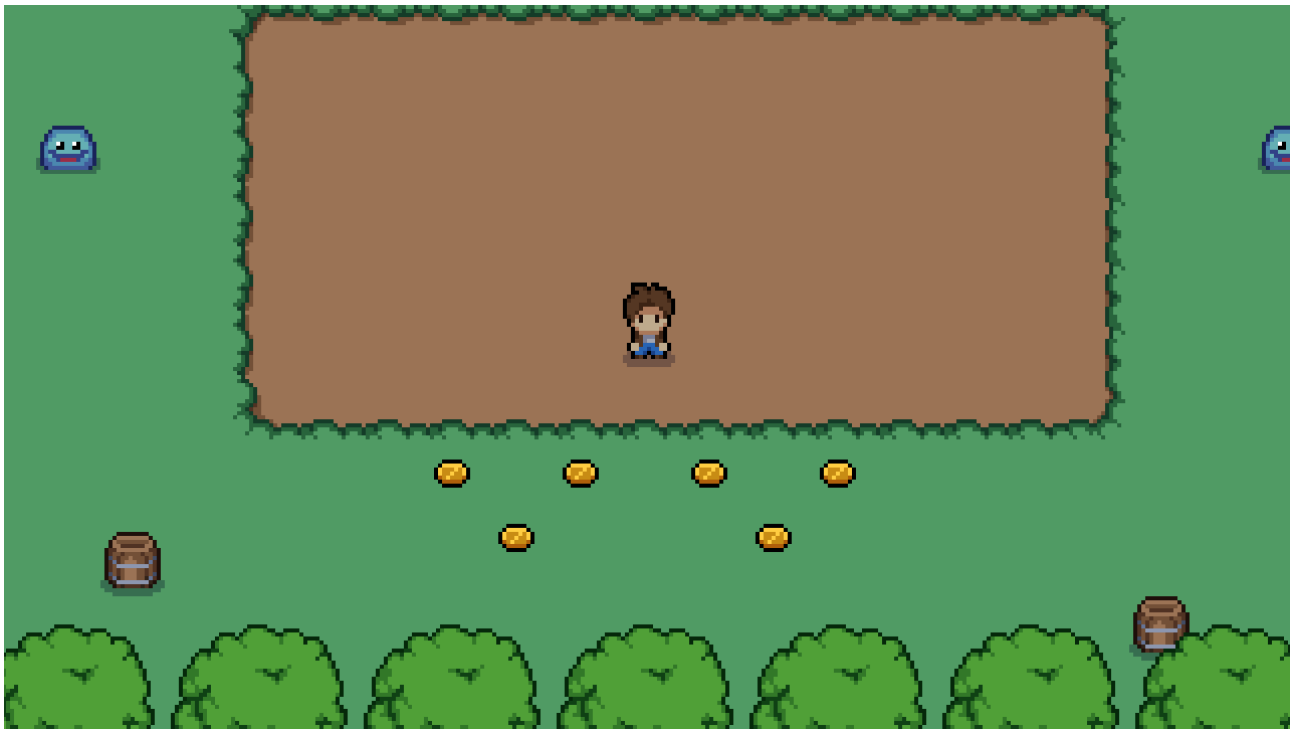


Figure 1: The finished scene: the player in the effects arena with slime dummies, barrels, coins and gems.

Before you start

1. Unzip `CRE132_E01_ParticleEffects_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_E01_ParticleEffects_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Effects/E01_ParticleEffects`.
3. Press **Play**. Walk with WASD or the arrows, swing the sword with **J** or the left mouse button, and throw a fireball with the right mouse button or **K** (it goes towards the mouse). The slimes turn red and die after 10 hits, a barrel breaks in one, and coins and gems vanish when you touch them. It all works, but nothing *shows* it yet.

The scene has the **Player** (with a child called **Dust** at its feet), five **Slime Dummies**, three **Barrels** and some coins and gems. The **Dust** and **Sparkle Gold** effects are already made, in `Assets/_game/Prefabs/Effects`. You'll make **Hit Sparks** and **Explosion** yourself.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Dust at your feet: FootstepDust

First look at an effect that's already made. Expand the **Player** in the Hierarchy and select **Dust**. A Particle System is an *emitter*: it throws out small squares (particles), each of which lives a short time and then disappears. The Inspector shows it in sections called **modules**; the main ones are:

- The top module (**Dust**): **Start Lifetime** (how long each particle lives), **Start Speed**, **Start Size** and **Start Color**. Dust uses **Random Between Two Constants** for the first three, so no two puffs look the same.
- **Simulation Space: World.** Particles stay where they were made. With **Local** they would follow the player around, which looks wrong for dust.
- **Color over Lifetime** fades the alpha to 0, and **Size over Lifetime** shrinks each particle, so they fade away instead of popping out.
- **Renderer** has a **Sorting Layer** and **Order in Layer**, just like a Sprite Renderer. Dust is on **Objects**, under the player.

Dust has **Rate over Time** set to 0 in **Emission** and **Play On Awake** off: it makes nothing by itself. A script asks it for a puff while the player is moving.

Make a folder `Assets/_game/Scripts/E01_ParticleEffects` and create a script in it called `FootstepDust`:

```
// Little puffs of dust at the feet while the character is moving. The Particle System
// has no emission of its own; this script asks it for a puff every so often with Emit.
using UnityEngine;

public class FootstepDust : MonoBehaviour
{
    // A child at the feet with a Particle System (Emission off, Simulation Space: World).
    [SerializeField] private ParticleSystem dust;
    [SerializeField] private float timeBetweenPuffs = 0.15f;
    [SerializeField] private int particlesPerPuff = 2;

    private Rigidbody2D body;
    private float nextPuffTime;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
    }

    private void Update()
    {
        bool isMoving = body.linearVelocity.magnitude > 0.1f;
        if (isMoving && Time.time >= nextPuffTime)
        {
            dust.Emit(particlesPerPuff);
            nextPuffTime = Time.time + timeBetweenPuffs;
        }
    }
}
```

- **`linearVelocity.magnitude`** is the body's speed in any direction. Above 0.1 counts as moving; a tiny bit of drift doesn't.

- `dust.Emit(2)` makes two particles straight away, using all the settings in the modules. The timer (`nextPuffTime`) gives one puff every 0.15 seconds, the same pattern as S01's fire.

Add it to the **Player**. Drag the **Dust** child from the Hierarchy into the **Dust** slot. Leave **Time Between Puffs** at 0.15 and **Particles Per Puff** at 2.

Test it: press Play and walk about. Puffs appear at your feet and stay behind as you move. Stand still and they stop.

2. Make the Hit Sparks effect

Now make your own effect. Choose **GameObject > Effects > Particle System**, rename it **Hit Sparks**, and in its **Transform** set **Position** to 0, 0, 0 and **Rotation X** to **0** (Unity tilts new systems by −90, which would spray the sparks away from the 2D camera). Then set:

Module	Setting	Value
Hit Sparks (top)	**Duration**	0.5, **Looping** off
	Start Lifetime	Random Between Two Constants, 0.15 and 0.25
	Start Speed	Random Between Two Constants, 3 and 6
	Start Size	Random Between Two Constants, 0.06 and 0.12
	Start Color	pale yellow, hex FFFFB3
	Simulation Space	World
	Stop Action	Destroy
Emission	**Rate over Time**	0
	Bursts	click **+**: Time 0, Count 14, Cycles 1
Shape	**Shape**	Circle, **Radius** 0.1
Color over Lifetime	**Color**	tick it; FFFFB3 at the start, FF801A (orange) at the end; alpha 255 at the start, 0 at the end
Size over Lifetime	**Size**	tick it; a curve from 1 down to 0.2
Renderer	**Material**	`Pixel Particle` (in `Materials/Particles`)
	Sorting Layer	Foreground

The small arrow at the right of a setting switches it to **Random Between Two Constants**. In the colour gradient, the markers along the top are alpha (see-through) and the ones along the bottom are colour. For the size curve, click the curve and pick the falling preset at the bottom of the curve panel, then drag its right-hand point down to 0.2.

- **A Burst** makes all 14 particles at once instead of a steady stream: a hit is one moment, not a fountain.
- **Stop Action: Destroy** deletes the whole object when the last particle has gone. Every hit makes a new copy, so without it the Hierarchy would fill up with dead effects.
- **Foreground** draws the sparks on top of the slime, so you always see them.

Drag **Hit Sparks** from the Hierarchy into `Assets/_game/Prefabs/Effects` to make it a prefab, then delete it from the Hierarchy.

3. Playing an effect: ParticleBurst

Create `ParticleBurst` in the `E01_ParticleEffects` folder:

```
// Plays a one-off particle effect (a prefab) at this object. Call Play from a Unity
// Event, such as Health's On Damaged (hit sparks) or On Died (an explosion).
// The prefab's Particle System has Stop Action: Destroy, so it removes itself when it's
// finished.
using UnityEngine;

public class ParticleBurst : MonoBehaviour
{
    [SerializeField] private ParticleSystem burstPrefab;
    // Where it appears, measured from this object (sparks look best at the middle of the
    // body).
    [SerializeField] private Vector3 offset = new Vector3(0f, 0.4f, 0f);

    public void Play()
    {
        Instantiate(burstPrefab, transform.position + offset, Quaternion.identity);
    }
}
```

- **`Instantiate`** makes a new copy of the prefab at the slime's position plus the offset. **Play On Awake** is ticked on the prefab, so it bursts as soon as it appears.
- **`Play` is public** and takes no values, so a **UnityEvent** in the Inspector can call it. Health doesn't need to know anything about particles.

Add it: select all five **Slime Dummies** and add **ParticleBurst**. Drag the **Hit Sparks** prefab from the Project window into **Burst Prefab**; leave **Offset** at 0, 0.4, 0. Then, one slime at a time, find **On Damaged ()** on its **Health** component. Click **+** to add an entry, then drag that same **Slime Dummy** into the object slot of the entry, then choose **ParticleBurst > Play ()** from the function menu.

Test it: press Play and hit a slime with the sword or a fireball. Yellow sparks spray out on every hit, and the copies disappear from the Hierarchy half a second later.

4. The Explosion

Make a second effect the same way (**GameObject > Effects > Particle System**, Position 0, 0, 0, Rotation X 0), called **Explosion**. It uses the same settings as Hit Sparks except:

Module	Setting	Value
Explosion (top)	**Start Lifetime**	0.36 and 0.6
	Start Speed	1.5 and 5
	Start Size	0.15 and 0.4
	Start Color	orange, hex FFD94D
Emission	**Bursts**	Count **40**
Shape	**Radius**	0.3
Color over Lifetime	**Color**	FFD94D at the start, grey 4D4D4D at the end (alpha 255 to 0)

More, bigger, slower particles that turn grey read as an explosion with smoke instead of a spark. Save it as a prefab in `Prefabs/Effects` and delete it from the Hierarchy.

Add it: select all three **Barrels**, add **ParticleBurst** and drag **Explosion** into **Burst Prefab**. This time use **On Died ()** on each barrel's **Health**: click **+**, drag the **Barrel** into the new entry and choose

ParticleBurst > Play (). (On Damaged would work too, but a barrel has only 1 health, and you want the explosion to mean “destroyed”.)

Test it: break a barrel. It vanishes in an orange burst that fades to grey.

> **Why the explosion survives the barrel:** Health destroys the barrel straight after On Died. The explosion is a new object of its own, not a child of the barrel, so it carries on.

5. Pickup sparkles: change ItemPickup

Coins and gems use T09’s `ItemPickup`. Open `Assets/_game/Scripts/T09_PickupsAndInventory/ItemPickup.cs`. Add this field just below `amount`:

```
    // Optional (E01): a particle effect left behind when it's picked up, such as P07's
    // sparkles.
    [SerializeField] private GameObject pickupEffect;
```

Then, in `OnTriggerEnter2D`, add the `if` block between the `for` loop and `Destroy`, so the end of the method reads:

```
        for (int i = 0; i < amount; i++)
        {
            inventory.Add(item);
        }
        if (pickupEffect != null)
        {
            Instantiate(pickupEffect, transform.position, Quaternion.identity);
        }
        Destroy(gameObject);
```

- **The effect is optional:** if the slot is empty, `pickupEffect` is `null` and nothing extra happens, so pickups in other scenes still work.
- **It’s made before `Destroy`** while the pickup still knows its own position. Like the explosion, it’s a separate object, so it outlives the coin.

Add it: select every **Coin Pickup** and **Blue Gem Pickup** and check the new **Pickup Effect** slot. If it’s empty, drag in the **Sparkle Gold** prefab from `Prefabs/Effects`.

Test it: walk over a coin. A burst of gold sparkles appears and falls where it was.

Check your work

- ☐ Dust puffs appear at your feet while you walk and stop when you stand still.
- ☐ Every hit on a slime sprays yellow sparks on top of it.
- ☐ A broken barrel bursts into orange squares that turn grey and fade.
- ☐ Coins and gems leave a sparkle when you pick them up.
- ☐ After a while in Play mode, no old Hit Sparks or Explosion copies are left in the Hierarchy.
- ☐ Compare with `CRE132_E01_ParticleEffects_Finished` if something behaves differently.

Key points

- **Particles are cheap, quick feedback.** Most effects are a burst of a few dozen small squares that fade and shrink.
- **Let effects clean up after themselves** with **Stop Action: Destroy**, or the Hierarchy fills up.
- **Simulation Space: World** keeps particles where they were made; **Local** drags them along with the emitter.

- **Events keep scripts separate.** Health says "I was damaged"; ParticleBurst decides what that looks like. You can swap the effect without touching Health.

If something goes wrong

- **The sparks are invisible:** check the Transform's **Rotation X** is 0, the **Renderer**'s **Sorting Layer** is **Foreground**, and the **Material** is **Pixel Particle**.
- **Sparks keep spraying like a fountain: Looping** is still ticked, or **Rate over Time** isn't 0.
- **Copies stay in the Hierarchy after the effect ends: Stop Action** isn't set to **Destroy** on the prefab.
- **Hitting a slime does nothing new:** the **On Damaged** entry has no object in its slot, or its function still says **No Function**. Choose **ParticleBurst > Play ()**.
- **UnassignedReferenceException in ParticleBurst or FootstepDust:** the **Burst Prefab** or **Dust** slot is empty.
- **The dust follows the player around:** the Dust's **Simulation Space** is **Local**.

Try it

1. Make the explosion throw debris that falls: set **Gravity Modifier** to 1, or tick **Noise** for a wobbly smoke.
2. Give the fireball a trail: add a looping Particle System as a child of the Fireball prefab's sprite (keep **Simulation Space: World** so the trail stays behind).
3. Make a blue sparkle for the gems: duplicate **Sparkle Gold**, change its colours, and use it on the **Blue Gem Pickups**.
4. In a platformer, play the same dust burst when the player lands (P06 has a *Land* moment you could call it from).